

Tetris Mix Multiplayer

López Arteaga, César Omar
Pumayauli Evangelista, Bryan Jose

Universidad Nacional de Ingenieria - Facultad de Ciencias.
Ciencias de la Computacion

Abril 2026

Outline

Desarrollo - Implementacion - Pruebas

Clase en lado del SERVIDOR

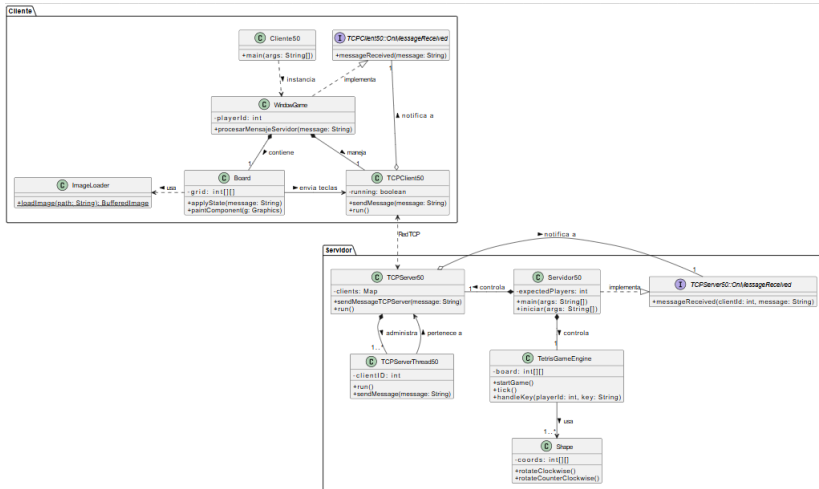
1. Servidor50
2. TCPServer50
3. TCPServerThread50
4. TetrisGameEngine
5. Shape

Clase en lado del CLIENTE

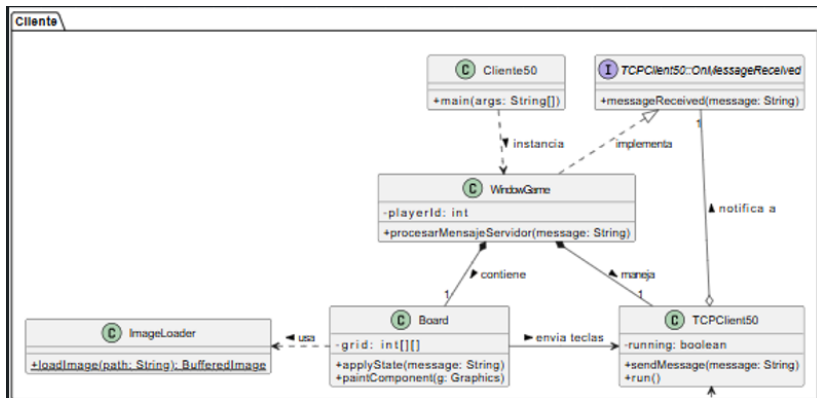
1. Cliente50
2. WindowGame
3. Board
4. TCPClient50
5. ImageLoader

Diagrama de Caso (UML)

Cliente - Servidor



Cliente



Servidor

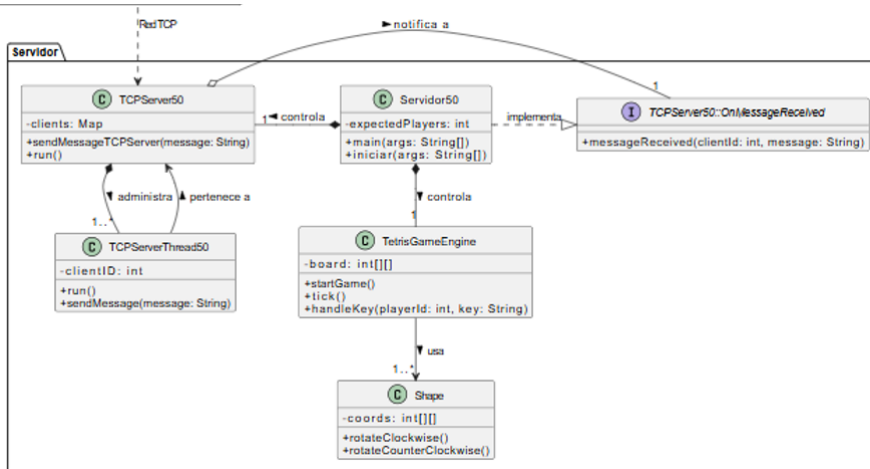
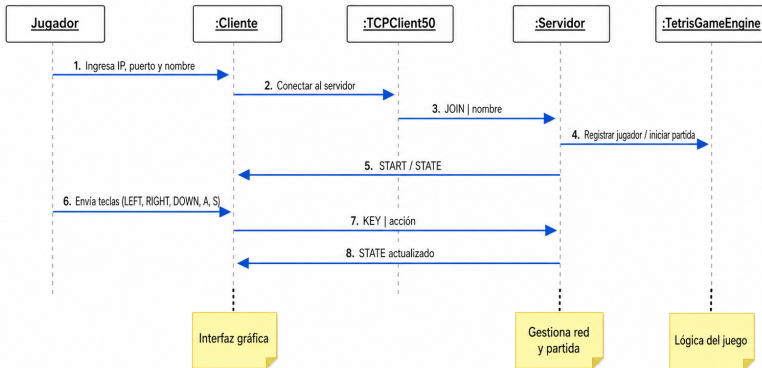


Diagrama de Comunicación

Diagrama de Comunicación Cliente-Servidor

Tetris Multijugador



Secuencia de trabajo del SERVIDOR

1. El servidor solicita el **puerto**, el **número de jugadores** y el tamaño del tablero.
2. Se crea el motor del juego:

`TetrisGameEngine(rows, cols)`

3. Se inicia el servidor TCP:

`TCPServer50(port, listener)`

4. El servidor queda escuchando conexiones de clientes mediante sockets TCP.
5. Cuando un cliente se conecta, se le asigna un identificador:

$J_1, J_2, J_3, \dots$

6. El servidor envía un mensaje de bienvenida al cliente:

`WELCOME|id|mensaje`

Secuencia de trabajo del SERVIDOR

7. Cuando el cliente envía:

`JOIN|nombre`

el servidor registra al jugador en el motor del juego.

8. Si ya se alcanzó el número esperado de jugadores, el servidor inicia la partida.
9. Durante el juego, el servidor recibe acciones del cliente:

`KEY|LEFT, KEY|RIGHT, KEY|DOWN, KEY|ROT_CW, KEY|ROT_CCW`

10. El motor actualiza el estado del tablero y el servidor envía a todos los clientes:

`STATE|...`

11. Un temporizador ejecuta periódicamente el avance automático del juego mediante:

`tick()`

12. Al finalizar, el servidor detiene el temporizador, cierra las conexiones y termina la ejecución.

Secuencia de trabajo del CLIENTE

1. El cliente solicita al usuario la **IP del servidor**, el **puerto** y el **nombre del jugador**.

2. Se crea la ventana principal del juego:

```
WindowGame(ip, port, name)
```

3. La ventana crea el tablero gráfico:

```
Board(rows, cols)
```

4. Se crea el cliente TCP encargado de la comunicación:

```
TCPClient50(ip, port, listener)
```

5. El cliente intenta conectarse al servidor mediante un socket TCP.

6. Cuando la conexión se establece correctamente, se envía el mensaje:

```
JOIN|nombre
```

7. El cliente recibe mensajes del servidor:

```
WELCOME|..., START|..., STATE|..., INFO|...
```

Secuencia de trabajo del CLIENTE

8. La ventana procesa los mensajes recibidos y actualiza la interfaz gráfica.
9. El tablero recibe el estado del juego mediante:

`applyState(message)`

10. El jugador presiona teclas de control:

`LEFT, RIGHT, DOWN, A, S`

11. El tablero envía la acción al servidor:

`KEY|acción`

12. Cada vez que llega un nuevo estado, el tablero se redibuja con:

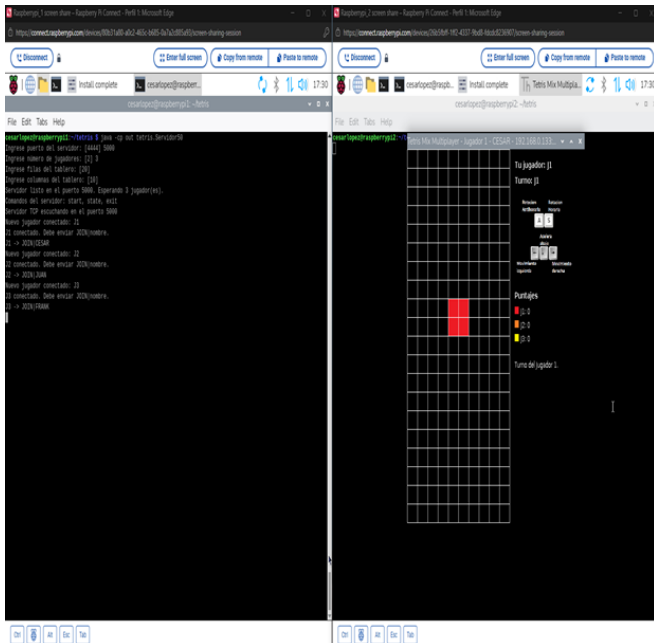
`repaint()`

Pruebas

Se hicieron las siguientes pruebas:

- ▶ En un computadora, se arranco el servidor y se trabajo en la misma computadora con 6 jugadores.
- ▶ En dos computadoras en la misma red, una computadora se uso como servidor y cliente, y la otra computadora exclusivamente como cliente.
- ▶ Se trabajo con 4 Raspberry pi 4B en una misma red, una se uso exclusivamente de servidor y los tres restantes de clientes

Pruebas



Pruebas

